Universidad Nacional del Altiplano — Escuela Profesional de Ingeniería de Sistemas
Algoritmos y Estructuras de Datos (SIS210) · Práctica N.º 07, sección VII (trabajo de investigación)

Hashing perfecto mínimo (MPH): algoritmos CHD y BDZ y su comparación con una tabla hash con rehashing

VILCA CHAÑI OLY AYMAR · PALACIOS HURTADO ROYER JOHAN · LLANOS VELASQUEZ CRISTIAN AUGUSTO · Docente: Dr. Aldo Hernán Zanabria Gálvez · Puno, octubre de 2026

Resumen. Este trabajo describe qué es una función hash perfecta mínima, en qué situaciones es práctico construirla, cómo funcionan los algoritmos CHD y BDZ y qué ofrecen las propuestas posteriores. Además, se implementó una versión didáctica de CHD en Python, verificada como biyección sobre $[0, n-1]$, y se la comparó con la tabla con encadenamiento y rehashing de la Actividad 2. Mi conclusión es que la MPH solo se justifica cuando el conjunto es estático, grande y mucho más leído que modificado; en los demás casos la tabla con rehashing es la mejor opción.

1. Definición y motivación

Una función hash perfecta (PHF) asigna a cada clave de un conjunto fijo $S \subseteq U$ un índice distinto en $[0, m-1]$; es *mínima* (MPHF) cuando $m = n = |S|$, es decir, cuando $f : S \to \{0, \ldots, n-1\}$ es una biyección (Pibiri y Trani, 2021). Con ella cada búsqueda cuesta un solo acceso y el arreglo de datos no tiene celdas vacías. La tabla de esta práctica no puede garantizarlo: aun con $\alpha \leq 0.75$, en el miniestudio de la Actividad 4 el encadenamiento tuvo colisiones en el 29.3 % de las inserciones con $\alpha = 0.75$, y el análisis clásico prevé $1 + \alpha/2$ comparaciones en una búsqueda exitosa (Cormen et al., 2022; Knuth, 1998).

El precio es doble. Primero, S debe conocerse de antemano. Segundo, la función solo tiene sentido sobre S: con una clave ajena devuelve un valor arbitrario en $[0, n-1]$ (Pibiri y Trani, 2021), de modo que, si llegan consultas externas, hay que comparar con la clave guardada o almacenar una huella. Además, guardar una MPHF exige como mínimo unos 1.44 bits por clave (Esposito et al., 2020); la pregunta práctica es cuánto se acerca cada algoritmo a esa cota y a qué costo de construcción.

2. ¿Cuándo es práctico construirla?

Las construcciones clásicas son estáticas y el caso dinámico es un problema aparte (Czech et al., 1997), así que el MPH solo se considera cuando se cumplen varias condiciones a la vez:

- **Conjunto estático o casi:** cualquier cambio de S obliga a reconstruir la función.
- **Muchas lecturas por construcción:** los buscadores y las bases de datos asignan identificadores compactos a conjuntos estáticos de cadenas (Pibiri y Trani, 2021); también se usan para reemplazar claves compuestas por claves sustitutas, acelerar uniones por hash y construir filtros con huellas (Botelho et al., 2007).
- **Memoria escasa:** la función ocupa pocos bits por clave y puede residir en caché mientras claves y datos quedan en memoria lenta (Belazzougui et al., 2009).
- **Escala:** para conjuntos masivos existen implementaciones paralelas que construyen la función para 10^{10} elementos en menos de 7 minutos con 8 hilos y 5 GB de memoria (Limasset et al., 2017).

No es práctico para conjuntos pequeños, para datos que cambian o cuando se necesita rechazar con rapidez claves que no pertenecen a S.

3. Algoritmos

3.1 Antecedentes

Fredman et al. (1984) mostraron que se puede almacenar un conjunto estático en $O(n)$ palabras con búsqueda de tiempo constante en el peor caso, usando dos niveles: una función de primer nivel reparte las claves en cubetas y cada cubeta recibe una tabla secundaria sin colisiones. Es perfecta, pero no mínima. Majewski et al. (1996) propusieron una familia de métodos basados en grafos aleatorios, y Czech et al. (1997) ofrecen una revisión completa de estos métodos, entre ellos el paradigma de mapeo, ordenamiento y búsqueda que muchos algoritmos posteriores siguen de una u otra forma.

3.2 CHD (compress, hash and displace)

Belazzougui et al. (2009) combinan el esquema *hash-and-displace* con la compresión de una secuencia de índices de funciones hash. El procedimiento, que es el que implementé en `mph_chd.py`, es el siguiente:

1. Repartir las n claves en $r = \lceil n/\lambda \rceil$ cubetas con una función hash (aquí $\lambda = 5$ claves por cubeta en promedio).
2. Ordenar las cubetas de mayor a menor tamaño, porque las grandes son las más difíciles de colocar.
3. Para cada cubeta, buscar un *piloto* $p = 0, 1, 2, \ldots$ tal que las posiciones $\mathrm{pos}(x, p) = \mathrm{mix}(h(x) \oplus pK) \bmod m$ de todas sus claves sean distintas entre sí y estén libres; se guarda p.
4. Evaluar $f(x)$ con un solo acceso: se calcula la cubeta de x, se lee su piloto y se aplica $\mathrm{pos}(x, p)$.
5. Para que el rango sea exactamente $[0, n-1]$, se usa $m = \lceil n/\alpha \rceil$ con $\alpha = 0.99$ y las pocas posiciones $\geq n$ se redirigen a los huecos libres menores que n con un arreglo auxiliar.

Según sus autores, la construcción es de tiempo $O(n)$, la evaluación es $O(1)$ y la representación es muy cercana al óptimo: 0.67 bits por clave para $m = 2n$ y 1.4 bits por clave para $m = 1.23n$ (Belazzougui et al., 2009).

3.3 BDZ

Botelho et al. (2007) siguen otro camino: tres funciones hash convierten cada clave en una arista de un hipergrafo 3-uniforme con algo más de n vértices. Con alta probabilidad el hipergrafo es acíclico, de modo que se puede ir quitando aristas con un vértice libre y, en el orden inverso, asignar a los vértices números pequeños (2 bits) que permiten elegir, para cada clave, cuál de sus tres vértices le pertenece; el índice mínimo se obtiene contando los vértices ya asignados. El método produce funciones de poco más de 3 bits por clave y los autores lo escalaron a más de mil millones de URL en algo más de una hora en un PC común (Botelho et al., 2007).

3.4 Propuestas más recientes

Tabla 1: Comparación de algoritmos según lo informado por sus autores.

Algoritmo	**Idea central**	**Espacio informado**	**Comentario**
FKS (1984)	dos niveles, tablas secundarias sin colisión	$O(n)$ palabras	perfecta, no mínima
BDZ (2007)	hipergrafo 3-uniforme acíclico	algo más de 3 bits/clave	$> 10^9$ URL en $\approx$1 h
CHD (2009)	cubetas, pilotos y compresión	1.4 bits/clave ($m = 1.23n$)	construcción $O(n)$
BBHash (2017)	niveles de vectores de bits; colisiones pasan al siguiente nivel	3.7 bits/elemento	10^{10} claves en < 7 min
RecSplit (2020)	división recursiva y biyecciones por fuerza bruta	1.56 bits/clave (8.3 % sobre la cota)	< 2 ms por clave
PTHash (2021)	pilotos con XOR (marco FCH)	competitivo en espacio	búsqueda 2–4$\times$ más rápida

Limasset et al. (2017) retomaron un método simple para lograr construcción rápida y poca memoria a escala masiva; RecSplit fue la primera estructura práctica en bajar de 2 bits por clave (Esposito et al., 2020); y PTHash recupera el marco de FCH con un esquema de pilotos que reduce el espacio y acelera las búsquedas entre 2 y 4 veces frente al estado del arte (Pibiri y Trani, 2021).

4. Experimento propio: MPH frente a la tabla con rehashing

Medí mi implementación de CHD ($\lambda = 5$, $\alpha = 0.99$, FNV-1a de 64 bits como hash base) con claves aleatorias de 8 letras minúsculas (semilla 42, Python 3.12, una sola ejecución) y la comparé con la `TablaHashChaining` de la Actividad 2. La biyección sobre $[0, n-1]$ se verificó para $n = 10$, 10^2, 10^3 y 10^4.

Se observan cuatro hechos. (1) El tiempo de construcción crece casi linealmente (cada $\times 10$ en n multiplica el tiempo por 9.2 y 9.5), de acuerdo con lo informado para CHD, pero cuesta entre 58 y 136 veces más que

Tabla 2: Resultados medidos. La columna "entropía" es una estimación (entropía de orden cero de los pilotos), no una codificación real.

	MPH (CHD)			**Tabla con rehashing (Act. 2)**			
n	construc. (s)	bits/clave (fijo)	bits/clave (entrop.)	construc. (s)	α final	celdas vacías	comp. éxito
10^3	0.37	2.91	—	0.003	0.63	53.4 %	1.32
10^4	3.40	3.14	1.82	0.025	0.39	67.8 %	1.19
10^5	32.33	3.37	1.99	0.557	0.49	61.5 %	1.19

construir la tabla con rehashing. (2) La descripción ocupa unos 3 bits por clave con ancho fijo, y cerca de 1.8–2.0 bits si se comprimiera como propone CHD. (3) El parámetro λ intercambia tiempo por espacio: con $n = 2 \cdot 10^4$, subir λ de 3 a 6 elevó el tiempo de construcción de 0.52 s a 26.3 s y redujo el espacio de 3.82 a 3.15 bits/clave (de 2.17 a 1.73 por entropía). (4) Entre 53 % y 68 % de las celdas de la tabla con rehashing quedan vacías, mientras que la MPH no deja ninguna.

En velocidad de búsqueda, en Python la MPH no gana: tardó 2.3–2.7 μs con FNV y 1.2–1.3 μs con el hash nativo, frente a 0.2–1.0 μs del encadenamiento. Atribuyo esa diferencia al intérprete (la MPH ejecuta en bytecode dos mezclas y un módulo, y el `hash()` nativo de la tabla corre en C), no a la estructura; por eso no extraigo conclusiones de velocidad a partir de este experimento.

5. Conclusión: ¿cuándo justifica la MPH su costo?

La tabla con rehashing de esta práctica admite inserciones y eliminaciones, se construye casi sin esfuerzo y ofrece unas $1 + \alpha/2$ comparaciones por búsqueda, pero desperdicia celdas y no da garantía de un solo acceso. La MPH garantiza un acceso y un arreglo denso con descripciones de pocos bits por clave, pero exige conocer S, cuesta mucho más construirla (aquí, dos órdenes de magnitud) y debe rehacerse si S cambia. Por tanto, el costo se justifica cuando se cumplen simultáneamente: el conjunto es estático, es grande, las consultas superan ampliamente a las reconstrucciones y la memoria o la caché son el cuello de botella. Para conjuntos pequeños o cambiantes, la tabla con rehashing sigue siendo la elección correcta.

Referencias

Belazzougui, D., Botelho, F. C., & Dietzfelbinger, M. (2009). Hash, displace, and compress. En A. Fiat & P. Sanders (Eds.), *Algorithms – ESA 2009* (pp. 682–693). Springer. https://doi.org/10.1007/978-3-642-04128-0_61

Botelho, F. C., Pagh, R., & Ziviani, N. (2007). *Perfect hashing for data management applications* (arXiv:cs/0702159). arXiv. https://doi.org/10.48550/arXiv.cs/0702159

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press. https://mitpress.mit.edu/9780262046305/

Czech, Z. J., Havas, G., & Majewski, B. S. (1997). Perfect hashing. *Theoretical Computer Science, 182*(1–2), 1–143. https://doi.org/10.1016/S0304-3975(96)00146-6

Esposito, E., Graf, T. M., & Vigna, S. (2020). RecSplit: Minimal perfect hashing via recursive splitting. En *2020 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)* (pp. 175–185). SIAM. https://doi.org/10.1137/1.9781611976007.14

Fredman, M. L., Komlós, J., & Szemerédi, E. (1984). Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM, 31*(3), 538–544. https://doi.org/10.1145/828.1884

Knuth, D. E. (1998). *The art of computer programming, Vol. 3: Sorting and searching* (2.ª ed.). Addison-Wesley.

Limasset, A., Rizk, G., Chikhi, R., & Peterlongo, P. (2017). Fast and scalable minimal perfect hashing for massive key sets. En C. S. Iliopoulos, S. P. Pissis, S. J. Puglisi, & R. Raman (Eds.), *16th International Symposium on Experimental Algorithms (SEA 2017)* (Art. 25). Schloss Dagstuhl. https://doi.org/10.4230/LIPIcs.SEA.2017.25

Majewski, B. S., Wormald, N. C., Havas, G., & Czech, Z. J. (1996). A family of perfect hashing methods. *The Computer Journal, 39*(6), 547–554. https://doi.org/10.1093/comjnl/39.6.547

Pibiri, G. E., & Trani, R. (2021). PTHash: Revisiting FCH minimal perfect hashing. En *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM. https://doi.org/10.1145/3404835.3462849